v1.0.0
20260311

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.4]
ELEMENTS OF CONCURRENT PROGRAM DESIGN

GUIDELINES FOR DESIGNING CONCURRENT PROGRAMS

- Process organised in two macro steps
  - **step #1 - analysis**
    - identifying tasks
  - **step #2 - design**
    - concurrent architectures

STEP #1 - ANALYSIS

- Analyzing the problem to identify exploitable concurrency
  - identifying ***task*** & ***data*** *decomposition* and their ***dependencies***
- **Task** definition
  - sequence of instructions that operate together as a group
    - corresponding to some logical part of an algorithm or a program
  - a specific piece of work required to be done as a duty or responsibility of some agent
    - more abstract / design-oriented definition

STEP #2 - DESIGN

- Choosing a suitable **concurrent architecture**
  - assigning tasks to "*agents*"
    - *active components* responsible of performing tasks
    - encapsulate the logic & control of task execution
  - in order to work together, agents share and use some *passive components*
    - *shared resources* encapsulating result of agent work
    - *coordination media/resources* to shape agent interaction and manage task dependencies

ACTIVE VS. PASSIVE COMPONENTS DISCIPLINE

- Strong partition
  - **active components ("agents")**
    - encapsulating a thread of control
    - no public interface
    - every interaction enabled and mediated by shared passive components
  - **passive components**
    - designed as monitors
- No hybrid entities
  - an hybrid entity can be designed as a set of one or more active and passive components

STEP #1 - PROBLEM ANALYSIS

- Objective:
  - identifying task / data decomposition and their dependencies
- Two steps
  - **Task** and **Data** Decomposition Analysis
  - Dependency Analysis

TASK DECOMPOSITION PATTERN

- Following a *divide-at-impera* principle, the problem can be naturally decomposed into a collection of independent or nearly independent tasks
  - tasks must be sufficiently independent so as that managing dependencies takes only a small fraction of the program's overall execution time
- First tasks are identified, then how data can be decomposed given the task decomposition
  - in this case, data decomposition follows task decomposition
- Example
  - producers/consumers systems

DATA DECOMPOSITION PATTERN

- Problem's data can be easily decomposed into units that can be operated on relatively independently
  - first data units are identified, then task related to that units can be identified
  - in this case, task decomposition follows data decomposition
- Scale well with the available number of processors
- Example
  - matrix multiplication

DEPENDENCY ANALYSIS

- Analyzing dependencies among the tasks within a problem's decomposition
  - to ***group*** and **order** tasks according to the type of dependencies
  - in order to maximize performances
- Possible kinds of dependencies
  - **temporal** dependencies
  - **data / resource** dependencies
- A classification of dependencies in coordination problem can be found in [MAL-93]
  - interdisciplinary issue

STEP #2 - DESIGN LEVEL - SOME CONCEPTUAL CLASSES

- From analysis to design: basic architectural classes proposed in literature [CAR-89]
  - **Result parallelism**
    - envisioning parallelism in terms of program's *result*
  - **Specialist parallelism**
    - envisioning parallelism in terms of an ensemble of specialists (the *crew*) that collectively constitute the programs
  - **Agenda parallelism**
    - envisioning parallelism in terms of program's *agenda* of tasks

RESULT PARALLELISM

- Designing the system *around the data structure or resource* yielded as the ultimate result
  - we get parallelism by computing all the elements of the result simultaneously
  - strictly related to the data-decomposition pattern
- Each agent is assigned to produce one piece of the result
  - they all work in parallel up to the natural restriction imposed by the problem
- Proper shared data structures are designed to wrap the result (data) under construction
  - encapsulating mutex and synchronization issues

RESULT PARALLELISM: "BUILDING A HOUSE" EXAMPLE

- Think about the components of the house
  - front, rear, side walls, interior walls, foundations, roof, etc.
- Proceed by building all the components simultaneously
  - assembling them as they are completed
- Use separate agents (workers) set to work on the different parts
  - proceeding in parallel up to the point where work on one component can't proceed until another is finished

SPECIALIST PARALLELISM

- Designing the system around an *ensemble of specialists* connected into a logical network of some kind
  - parallelism results from all the nodes of the logical network (all the specialists) being active simultaneously
- Each agent is assigned to perform one specified kind of task
  - they all work in parallel up to the natural restriction imposed by the problem
- Coordination means (communication protocols, shared data structures) are used to support specialists communication and coordination
  - examples: message boxes, blackboards, event services
- Opposite approach with respect to result parallelism

SPECIALIST PARALLELISM: "BUILDING A HOUSE" EXAMPLE

- Identify the separate skills required to build the house
  - surveyors, excavators, foundation builders, carpenters, roofers, etc
  - different kind of tasks
- Then assemble a constructor *crew* in which each skill is represented by a separated specialist worker
- They all start simultaneously
  - but initially most workers will have to wait around
  - once the project is well underway, however, many skills (hence many workers) will be called to play simultaneously
- Remarks
  - strong role of worker interaction and coordination
    - different kind of strategies
    - e.g. pipelined jobs

AGENDA PARALLELISM

- Designing the system around a particular **agenda of tasks** (activities in [CAR-89]) and then assign many workers to each step
  - and also possibly carry on different steps in parallel
    - *workflow*
- Each agent is assigned to help out with the current item on the agenda
  - they all work in parallel up to the natural restriction imposed by the problem
- Shared data structures are designed to manage data consumed and produced by agenda activities
  - examples: bounded-buffers

AGENDA PARALLELISM: "BUILDING A HOUSE" EXAMPLE

- We write out a sequential agenda and carry it in order, but at each stage we assign many workers to the current activity
  - doing the foundation, doing the frame, doing the roof, etc
- We assemble a work team of **generalists**
  - each capable of performing any construction step
- The team is set to work stage by stage following the agenda

REMARKS

- The boundaries between the three classes are not rigid
  - we will often mix elements of several approaches in getting a particular job done
    - e.g. a specialist approach might make secondary use of agenda parallelism, for example, by assigning a team of workers to some speciality
- However the three approaches represent *three clearly separate ways of thinking about the problem*
  - result parallelism focuses *on the shape of the finished product*
  - specialist parallelism focuses *on the makeup of the work crew*
  - agenda parallelism focuses *on the list of tasks to be performed*
- The approach can be recursively applied
  - following task and data decomposition

USING CONCEPTUAL CLASSES - BASIC STEPS

- To write a concurrent program (adapted from [CAR-89]):
  - choose the concept class that is most natural for the problem;
  - write a program using the method that is most natural for that conceptual class; and
  - if the resulting program is not acceptably efficient, transform it methodically into a more efficient version by switching from a more-natural method to a more-efficient one

STEP #2: CONCURRENT ARCHITECTURES

- Some specific architectures can help to bridge the gap between the conceptual class and the implementation (adapted from [MAG-99][PAT-06])
  - **Master-Workers (or Manager-Workers)**
    - Task-oriented frameworks
    - Fork-Join variant
  - **Filter-Pipeline**
    - Stream-based patterns
  - **Shared-Space / Blackboard**
    - Space-based coordination patterns
  - **Announcer-Listeners**
    - Event-based coordination patterns

MASTER-WORKERS

- Description
  - the architecture is composed by a *master* agent and a possibly dynamic set of *worker* agents, interacting through a proper coordination medium functioning as a **bag of tasks**
  - **master agent**
    - decompose the global task in subtasks
    - assign the subtasks by inserting their description in the bag
    - collect tasks result
  - **worker agent**
    - get the task to do from the bag, execute the tasks and communicate the results
      - variant: proper monitor data structures can be used to aggregate results
  - **bag of tasks resource**
    - typically implemented as a blackboard or a bounded buffer
- Example of problems
  - quadrature problem

TASKS AND ASYNC PROGRAMMING

- The execution of a task occurs *asynchronously* with respect to the control flow that created and submitted the task
  - already discussed in previous module
- Example: **Java Executor framework**
  - master-worker architecture
  - a task is submitted to an executor by means of an execute or submit method, which returns immediately
    - a *future* can be returned
  - the task is *asynchronously* executed by one of the executor threads, according to the executor policy

TASK-ORIENTED FRAMEWORKS

- **Task** concept
  - abstract, discrete, independent *unit of work*
    - domain/problem level
  - uncoupled from the notion of thread
    - it will be executed by some thread or process of the system
- Task-oriented analysis & design
  - analyzing the problem to identify exploitable concurrency
  - identifying task & data decomposition and their dependencies
- Tasks as programming abstraction
  - higher-level than threads
    - logic concurrency vs. physical concurrency
  - frameworks/libraries example: Java Executors

TASKS AND ASYNC PROGRAMMING

- In task-oriented frameworks, there are two distinct roles:
  - who *submits* the task
    - playing the role of master
  - who *executes* the task
    - worker threads inside the framework
- Key point
  - the task is executed **asynchronously** with respect to the control flow of the task submitter
    - *task submission does not block waiting the task to be completed*
  - follow-up problem: *how the master can collect the task results (computed asynchronously)?*
    - through a bag of results
    - through a specifically-designed collector/aggregator monitor
    - **future mechanism**

FUTURE MECHANISM

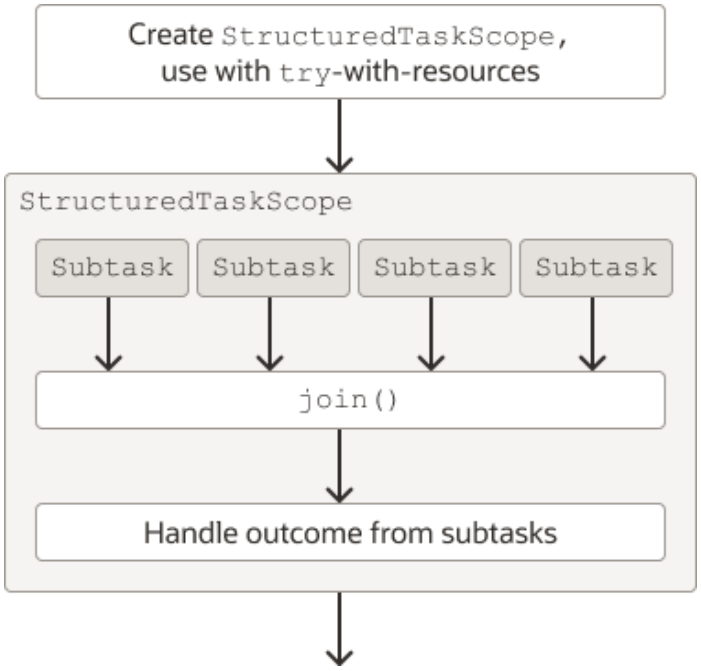
- Future mechanism
  - by issuing an asynchronous computation (task), an object *representing the future result* or the state of the computation itself is created and immediately returned
- The object could provide operation for
  - checking the state of the task (*poll*)
  - blocking when the task result is needed
  - cancel the ongoing task, when possible
  - catching errors/exception related to the ongoing task
- **Important remark**
  - with task+future, we have async programming *but still allowing for blocking behaviour*
  - in event-driven programming (next slides) blocking behaviour is no more allowed

TASKS & VIRTUAL THREADS

- Tasks promote a higher-level logical view about concurrency
  - vs. physical view given by (OS, physical) threads
- Actually, a similar logical view is provided by frameworks/languages/ platforms introducing a logical/lightweight notion of thread
  - also called user-level threads / fibers / green threads
  - uncoupled from physical threads (OS-level), used to run them
- In Java, this logical view is represented by **virtual threads** recently introduced (as a preview) in JDK 19
  - Java Enhancement Proposal (JEP) 425
    - https://openjdk.org/jeps/425
  - a main result of the project Loom
    - https://openjdk.org/projects/loom/
- Discussed in a module-lab

TASKS & STRUCTURED CONCURRENCY

- Simplify concurrent programming by introducing an API for **structured concurrency**
  - treats groups of related tasks running in different threads as a single unit of work
  - streamlining error handling and cancellation
  - improving reliability, and enhancing observability



- Proposed by Java Enhancement Proposal (JEP) 425
  - https://openjdk.org/jeps/425

FORK-JOIN

- Description
  - applying the master worker architecture recursively
    - workers are themselves masters, producing tasks and collecting results
  - effective approach to implement a concurrent divide-and-conquer strategies
  - can be realized either using using a bag of tasks or directly
- Example of problems
  - adaptive quadrature problem
  - merge-sort

FILTER-PIPELINE

- Description
  - the architecture is composed by a linear chain of agents interacting through some *pipe* or *bounded buffer* or *channel* resources
  - **generator agent**
    - the agent starting the chain, generating the data to be processed by the pipeline
  - **filter agent**
    - an intermediate agent of the chain, consuming input information from a pipe and producing information into another pipe
  - **sink agent**
    - the agent terminating the chain, collecting the results
- Example of problems
  - image processing

SHARED-SPACE

- Description
  - the architecture is composed by a set of agents communicating and indirectly interacting through a shared space, as a shared resource functioning as a blackboard-like coordination media
  - the space-based coordination media provides operations to add new information items and to query/retrieve/remove them, eventually including some synchronisation feature
- Features
  - data-driven coordination
  - temporal uncoupling
- Main reference in literature
  - Tuple spaces and Linda coordination language [GEL-85][GEL-92]

ANNOUNCER-LISTENERS

- Description
  - the architecture is composed by an *announcer* agent and a dynamic set of *listener* agents, interacting through an *event-service*
  - **announcer agent**
    - announce the occurrence of events on the event service
  - **listener agents**
    - register on the event service so as to be notified of the occurrence of events interesting for the listener
  - **event-service resource**
    - uncouple announcer-listeners interaction
    - collect and dispatch events
- Example of problems
  - parallelization of event-driven applications
    - achieved by parallelizing the execution of event handlers

REVISITING THE MODEL VIEW CONTROLLER /1

- Issue
  - how to separate model / view / controller aspects in the design of complex concurrent programs
- A first mapping
  - encapsulating control in agents
    - active components of the program
    - encapsulating the control logic
  - model as shared passive components
    - can be used by agents
    - encapsulating mutex properties
    - includes coordination media
  - view as shared passive components
    - enabling and mediating the interaction with users
    - observed and used by agents
    - can access to model resources to get the state

REVISITING THE MODEL VIEW CONTROLLER /2

- Issues / problems (to be discussed)
  - A common design choice in existing GUI toolkit is to adopt a single active component to be responsible of managing the GUI aspects, with all GUI components are not thread-safe
    - how to deal with it using the approach described so far?
  - More generally: MVC adopts an *event-based* approach to interactions
    - *observer pattern*
      - the original one is synchronous and involves control coupling
  - **Rethinking the observer pattern for concurrent systems**
    - producer/consumer architecture
      - bounded-buffer is an event queue
      - producers: who generate events
      - consumer: who reacts and consumes events

BIBLIOGRAPHY

- **[CAR-89]** Nicholas Carriero and David Gelernter, "How to write parallel programs: a guide to the perplexed", ACM Computing Surveys (CSUR). Volume 21, Issue 3 (1989)
- **[GEL-92]** David Gelernter and Nicholas Carriero. 1992. Coordination languages and their significance. Commun. ACM 35, 2 (Feb. 1992), 97–107. DOI:https://doi.org/10.1145/129630.129635
- **[GEL-85]** David Gelernter. 1985. Generative communication in Linda. ACM Trans. Program. Lang. Syst. 7, 1 (Jan. 1985), 80–112. DOI:https://doi.org/10.1145/2363.2433
- **[MAT-05]** T. Mattson, B. Sanders, B. Massingill, "Patterns for Parallel Programming", Addison Wesley (2005)
- **[MAG-99]** Jeff Magee and Jeff Kramer, "Concurrency - State Models and Java Programs", Wiley (http://www.doc.ic.ac.uk/~jnm/book/)
- **[MAL-93]** Thomas W. Malone, Kevin Crowston. "The interdisciplinary study of coordination". Center for Coordination Science, Alfred P. Sloan School of Management, Massachusetts Institute of Technology, 1993
- **[PAT-06]** Jorge Luis Ortega-Arjona. Patterns for Parallel Software Design. Wiley (2006)